Homework 2

Problem 1

The two major concerns of any software project are how much it will cost and how long it will take. I think time is slightly more important because falling behind schedule can affect the entire team and lead to additional costs. Complete functionality is also important because the goal is to deliver software that meets the customer's needs. However, trying to include every feature can increase both the time and cost of development. I think it's better to prioritize the most important features and make sure they work properly rather than rush to include everything and risk going over budget or missing deadlines.

Problem 2

The five main phases of Agile development are requirements gathering, design, development, testing, and feedback. I think some of the initial requirements gathering and overall design can be done at the beginning of the project to give the team a clear direction. For RealEye, having a general understanding of what we're building and how everything should work together helps us divide responsibilities and stay organized. However, I don't think any of these phases should be completely skipped in later iterations because requirements and design can change, and we might discover issues during development. While doing more planning upfront could save some time, I think repeating these phases ultimately saves more time by helping us catch problems early instead of having to fix bigger issues later.

Problem 3

In the Waterfall method, the main phases are requirements gathering, system design, implementation, testing, and maintenance. These phases are completed sequentially, meaning each phase is generally finished before moving on to the next. In contrast, Agile follows an iterative approach where planning, design, development, and testing happen repeatedly in shorter cycles, allowing for continuous customer feedback and changes. Waterfall includes more distinct, formal phases for upfront requirements gathering, system design, and maintenance, while Agile incorporates these activities throughout development rather than treating them as separate phases. I think these separate phases can be useful because they help establish clear requirements and a detailed plan before development begins, especially for large projects, while also providing a structured approach to maintaining the software after release. For example, if an Agile team were developing software for healthcare systems, a more formal upfront design and requirements phase might be necessary to ensure the system meets strict safety and regulatory requirements before implementation.

Problem 4

1. What is a user story?

A user story is a short description of how users interact with a feature or functionality of the software, written from the customer's perspective in a way that both the customer and developers can understand.

2. What is blueskying?

Blueskying is a brainstorming process where developers and customers think big and freely discuss ideas and possible features without initially worrying about technical limitations, cost, or feasibility.

3. What are four things that user stories SHOULD do?

User stories should describe one thing the software needs to do for the customer, use language the customer understands, be written by the customer, and be short, ideally no more than three sentences.

4. What are three things that user stories SHOULD NOT do?

User stories should not be long essays, use technical terms that are unfamiliar to the customer, or mention specific technologies.

5. Does the Waterfall method have user stories?

The traditional Waterfall method does not use user stories as a core part of its process, but instead relies on detailed requirements documents to define what the software should do.

Problem 5

I agree that all assumptions are bad because they can lead to misunderstandings and inaccurate estimates. I think it's important to clarify assumptions with the customer whenever possible rather than just assuming something is true. However, I don't think all assumptions can be avoided, especially when the customer doesn't have all the answers yet. In those cases, it's important to identify and keep track of those assumptions as potential risks.

I agree that a big user story estimate is a bad user story estimate because larger estimates are more likely to be inaccurate, especially since requirements can change over time. I think breaking larger user stories into smaller ones makes them easier to understand, estimate, and

complete within an iteration. It also helps the team identify potential problems earlier and make more realistic estimates.

Problem 6

1. You can dress me up as a use case for a formal occasion: **User story**
2. The more of me there are, the clearer things become: **User story**
3. I help you capture EVERYTHING: **Blueskying and Observation**
4. I help you get more from the customer: **Role playing and Observation**
5. In court, I'd be admissible as firsthand evidence: **Observation**
6. Some people say I'm arrogant, but really I'm just about confidence: **Estimate**
7. Everyone's involved when it comes to me: **Blueskying**

I agree with the answers on page 62. User stories help clarify requirements, while blueskying, role playing, and observation help uncover ideas and details that might otherwise be missed. The last answer makes sense because blueskying includes both the customer and the development team. Planning poker might seem like another possible answer, but the book points out that customers are not involved in that activity.

Problem 7

A better than best-case estimate is an estimate that allows even less time than the work would take under ideal conditions. For example, if a feature would take five days with clear requirements, no interruptions, and no problems, promising it in three days would be a better than best-case estimate. It creates an unrealistic expectation because it assumes the team can finish faster than even its most optimistic reasonable prediction.

Problem 8

I think the best time to tell the customer is as soon as there is enough evidence that the delivery schedule cannot be met. This gives both the customer and the development team enough time to plan accordingly and consider alternative options. Waiting until the deadline leaves the customer with fewer options and can damage their trust. It would probably be a difficult conversation because nobody likes giving disappointing news. I would make it easier by explaining the reason clearly, showing the progress so far, and bringing possible solutions, such as moving the deadline or delivering the most important features first.

Problem 9

I think branching is a good thing because it allows developers to work on different features and parts of the code at the same time without interfering with each other's work or affecting the main working version of the software. For example, in our RealEye project, we have different

teams working on the Chrome extension, backend server, and machine learning models. Having separate branches allows everyone to work on their own parts of the project without risking changes to the main branch. However, branching can also make development more complicated because merging changes can lead to conflicts, and each branch requires additional maintenance, testing, documentation, and upkeep. I think branching is useful as long as it's done carefully and branches are merged regularly to avoid unnecessary complications.

Problem 10

I have used Vite and npm scripts as build tools for many projects. I like that Vite makes it easy to run a development server and automatically updates the application when changes are made to the code. npm scripts are also useful because they allow us to run commands to build and test the project without having to manually enter multiple steps. This makes the development process more efficient and helps ensure everyone on the team follows the same process. However, configuring these tools and managing dependencies can sometimes be confusing, especially when errors occur.